

Hiranandani Dashboard

Application flow, end to end

React (Vite) + Express · Sperto CRM integration · generated 21 September 2026

This is what the app does, step by step, from a sales staff member signing in to the numbers landing in the client's CRM. Every behaviour described here was run against the code, not read off it — the payloads and timings shown are captured from real runs.

The shape of it

Three screens. One login is one customer. There is no “End Session” button — signing out is what closes a presentation, and that is deliberate: it means a presentation cannot be left half-ended, whichever way the staff member leaves.

```
/login          -> /session/start      -> /dashboard
email or        -> Lead ID + device   -> VR tour + projects
Sales ID        (both Sperto-checked) (timed per project)

Log out from any of them -> session closed, times sent to Sperto
```

Two systems own the lists

- Sperto (the client's CRM) owns who the staff are and who the customers are. Neither has to be pre-created in this app.
- This app owns what happened: every login, search, project opened and closed, with measured time. That log is append-only — there is no route that can edit or delete it.

1 · The flow, step by step

1 Sign in — /login

Staff type their email or their Sperto Sales ID (e.g. PDPL0349). No password. The server asks Sperto whether that identity is real. An identity they do not have is refused. Admins and managers use email + password on a separate door, deliberately not Sperto-gated so a CRM outage cannot lock them out.

Reporting is out of scope this release, so admin/manager sign-in returns 403.



2 Who are you, really? — route guard

The browser is handed a signed, httpOnly session cookie. Before any screen is drawn, the app asks the server to verify it (GET /api/session). The role comes from that verified token, never from the readable role cookie, which is display-only and could be edited by hand.



3 Find the customer — /session/start

The staff member types the Lead ID (or phone). It goes to Sperto too, through the same endpoint as the login, as type: "lead".

Sperto says no -> the session does not start.

Sperto is down -> fall through to our own table; a customer is waiting.

No Lead ID at all -> "Continue without a Lead ID" makes a walk-in record.



4 Pick the device

Tab / TV / Kiosk / Laptop, chosen by the staff member rather than guessed from the browser string — which can only ever say "Chrome · Windows", never "Kiosk".

This is what makes "which device sells the most" answerable.

Sperto is told the session started: an IN call fires here.



5 The presentation — /dashboard

The 360° VR tour plays behind everything. Every project sits in a rail over it.

Tapping one opens that project's own website full-screen, in this tab — not a new browser tab, so the session header, the timers and Log out stay reachable.

A per-project clock starts. Opening another project stops the first one.

Staff can add meeting notes and flag themselves Busy; both are logged.



6 Log out — the only way out

Reachable from the header, and from inside a project's full-screen viewer.

It also happens by itself: 30 minutes idle, an admin force-logout, or the same account signing in elsewhere. All six routes run the same code.

The presentation is closed out, and an OUT call carries the times to Sperto.

2 · What reaches Sperto

Two integrations, one host. Both were verified end to end against a stand-in server built to the CRM's own documented quirks.

The rule that governs both

Their server does not use HTTP status codes to mean anything.

a rejection	arrives as	HTTP 200	
a success	arrives as	HTTP 500	(device-usage, confirmed live)

So nothing branches on the status. Every answer is read out of the body, in one place, by both integrations.

This matters most for the device-usage log, which previously discarded the response entirely. That meant a visit Sperto refused to store was indistinguishable from one it stored — the only symptom being numbers quietly missing from the CRM. It now reports what actually happened, and logs a failure in Sperto's own words.

Session start — IN

```
POST api_record_device_usage.php
{
  device_id: "3",                <- Kiosk
  lead_id: "WALKIN-1789969341382",
  sales_manager_login: "PDPL0349", <- the Sales ID they signed in with
  type: "IN",
  page_url: ".../session/start"   <- nothing opened yet
}
```

Session end — OUT (this is the one that carries the times)

```
POST api_record_device_usage.php
{
  device_id: "3",
  lead_id: "WALKIN-1789969341382",
  sales_manager_login: "PDPL0349",
  type: "OUT",

  page_url: [                                <- their field, now an array
    { "Elena": 8 },
    { "Alibaug": 15 }
  ],

  project_time: { Elena: 8, Alibaug: 15 } <- the same, as one object
}
```

Both fields carry the same numbers on purpose, one object per project against one object for all of them. `project_time` is a custom field their published API does not list, so there is no guarantee their backend stores it. `page_url` is theirs and always has been. Whichever one they are actually reading has the numbers in it — and a test asserts the two agree second for second.

What the answer means now

Sperto's reply	We report	What it means
200, status: success	<code>recorded: true</code>	Stored.
500, status: success	<code>recorded: true</code>	Also stored — their host really does this.
200, status: error	<code>recorded: false · rejected</code>	They refused it. Logged with their message.
500, empty body	<code>recorded: false · unavailable</code>	Their known endpoint bug. Never read as “wrong user”.

The route still answers 200 / ok: true in every case, on purpose — a presentation must be free to start and to end whether or not the CRM accepted the write. The staff member has already walked away by the time an OUT answers.

3 · How time is measured

Per-project time is measured, not inferred from the gap between log entries — that guess is wrong the moment a session ends on a project.

The rules, each covered by a test

Rule	Why it matters
Only projects actually opened appear	A zero in the CRM reads as a visit that happened.
Reopening adds, never duplicates	Elena 3 min + Elena 2 min is one 300s entry, not two.
Two projects never run at once	Opening B banks A's time first, so a second is never counted twice.
A backgrounded tab accrues nothing	A session left open over lunch must not report an hour.
A refresh does not invent time	Time up to the reload is kept; everything after it would be made up.
The project on screen at logout counts	Closing the app is not the same as the customer leaving the room.
Exactly one OUT per presentation	Six exits share one code path; only the first gets to send.
One customer's time never lands on the next	Starting a session clears the slate.

Verified against a real run

A presentation was run in a browser. These are the two records it produced, side by side:

Project	Activity log (measured)	Sent to Sperto	Agree?
Elena	8,165 ms	8 s	yes
Alibaug	14,507 ms	15 s	yes

Alibaug was still open on screen when Log out was pressed — and it was still counted. That is the rule above, working.

The activity log

Separate from Sperto, and richer: every login, search, customer lookup, project open and close, tab viewed, note written, status toggle and logout, each stamped with the device the staff member chose. Append-only — the route has a GET and a POST and deliberately no PATCH or DELETE, so sales staff have no path to edit or erase their own history.

login	"Sales Staff signed in"	
step	"Entered presentation"	device=Kiosk
tour_view	"360° VR tour opened"	device=Kiosk
project_open	"Opened Elena"	device=Kiosk
project_close	"Closed Elena"	device=Kiosk (8165ms)
project_open	"Opened Alibaug"	device=Kiosk
project_close	"Closed Alibaug · logged out"	device=Kiosk (14507ms)
step	"Left presentation"	device=Kiosk (39472ms)
logout	"Sales Staff signed out"	

4 · Every way a session ends

Six exits, one code path. Before this, only the Log out button reported an OUT — so an idled-out session left Sperto believing the customer was still in the room.

Exit	Who triggers it	Reports OUT
Log out — showcase header	Staff	yes
Log out — inside a project viewer	Staff (new)	yes
Log out — customer lookup screen	Staff	yes
Log out — reports dashboard	Admin / manager	yes
Idle for 30 minutes	By itself, with a 60s warning	yes
Force-logout by an admin	Admin / manager	yes
Same account signs in elsewhere	By itself	yes

The Log out inside a project's full-screen viewer is new. That view covers the whole screen, including the header everything else signs out from — so ending a presentation from inside a project used to mean closing back out first, which on a kiosk with no browser chrome is the difference between one tap and being stuck.

What happens, in order

1. close the open project -> logs it, with its measured dwell time
2. close the presentation -> banks the final step's time
3. send OUT to Sperto -> page_url array + project_time
4. clear the session cookie -> server-side; the browser cannot do it
5. hard navigation to /login

The order is load-bearing. Both log writes are attributed to the active session, so they have to happen before it is cleared — otherwise that project's dwell time and the presentation's final step are both silently lost.

5 · What was checked

Every line below was run, not assumed.

Automated

● Unit tests	65 passed · Sperto response reading, per-project time accounting
● Production build	428 modules, clean
● Lint	0 errors (23 warnings, all pre-existing patterns)
● Secret leak check	no Sperto key anywhere in the built bundle

Live API — 39 checks, all passing

Area	Checked
Login	Sales ID accepted / refused, email accepted / refused, wrong password, missing identifier, admin door closed while reporting is off
Cross-site protection	POST with no Origin refused, POST from another origin refused
Session guard	signed in resolves, signed out refused, guard bounces /dashboard to /login
Lead ID	Sperto-known accepted with their name, Sperto-rejected blocks the session, outage falls through
Device usage	IN, OUT with visits, rejection, empty body, bad device type, bad call type, signed out
Activity log	append, read back, backend report, no PATCH, no DELETE
Admin-only	inventory write, user directory, account creation, customer deletion — all refused for staff
Backup	refused with no secret and with a wrong secret

In a real browser

- Signed in by Sales ID, through Sperto.
- A Lead ID Sperto does not have was refused on screen — “That Lead ID isn't registered in Sperto.” — and the session did not start.
- A walk-in session started when the lead database was unavailable, using the client-side fallback.
- Two projects opened and closed; the VR tour, the rail, notes and the Busy toggle all rendered and logged.
- Logged out from inside a project's full-screen viewer.
- The resulting Sperto payload and activity log were read back and matched to the second.
- No console errors at any point.

One bug found and fixed

Auditing the flow turned up a real defect carried over from the previous codebase: POST /api/leads opened a database connection before checking authorisation. On an instance with no database, a sales staff member attempting to delete a customer got “500 · DATABASE_URL is not set” — the wrong status and the wrong story, since the request should have been refused on authorisation, not plumbing. Authorisation is now checked first, and a no-database instance answers each action honestly.

6 · Still open with the client

None of these block the flow. All of them should be confirmed before the CRM goes live for real staff.

#	Question	Why it matters
1	Does their parser accept <code>page_url</code> as an array of <code>{ project: seconds }</code> ?	It previously received a single URL string. Their server ignores what it does not understand and answers 200 either way — so an old parser looks exactly like success from our side. (<code>project_time</code> is sent alongside for this reason.)
2	Is <code>project_time</code> stored on their side?	It is a custom field, not in their published API. Same invisibility problem as above.
3	What does a success body actually look like?	We have only ever been shown error bodies. Anything that is not an explicit failure word is read as success.
4	The real <code>device_id</code> per device type	Tab / TV / Kiosk / Laptop currently map to 1 / 2 / 3 / 4 — a placeholder guess.
5	Each staff member's Sperto login code	Without one, that account's sessions skip the device-usage call rather than send a made-up value.
6	Is the login endpoint's 500 fixed?	It was returning HTTP 500 with no body for any type value. Our side now survives it either way, but it must be re-tested before the key is enabled.

One consequence worth planning for

Turning Sperto on turns the demo customers off.

LEAD-1001 ... LEAD-1005 are ours, not theirs. Once the CRM is answering, it will refuse them — which is correct, and is the whole point of the check. Demos will need a real Lead ID from that moment on.